Partie 2 - Chiffrement et déchiffrement (ciphers.py)

```
import string
from math import gcd

ALPHA = string.ascii_uppercase
```

Outils communs

Les trois chiffrements travaillent sur l'alphabet A-Z. La fonction clean garde uniquement les lettres en majuscules.

```
def clean(text):
    return "".join(c for c in text.upper() if c in ALPHA)
```

Chiffrement de César

Chaque lettre est décalée de n rangs vers la droite ou la gauche. La casse et la ponctuation ne changent pas. Le déchiffrement applique le sens inverse. Deux attaques : essayer les 26 décalages, ou deviner le décalage avec la lettre la plus fréquente (le E).

```
def check_caesar_key(n):
    if n % 26 == 0:
        raise ValueError("clé invalide : un décalage multiple de 26 ne change pas le message")

def caesar(text, n, direction="right"):
    shift = n % 26
    if direction in ("left", "gauche"):
        shift = -shift
    result = ""
    for ch in text:
        if ch.isascii() and ch.isalpha():
            start = ord("A") if ch.isupper() else ord("a")
            result += chr((ord(ch) - start + shift) % 26 + start)
        else:
            result += ch
    return result

def caesar_decrypt(text, n, direction="right"):
    if direction in ("left", "gauche"):
        return caesar(text, n, "right")
    return caesar(text, n, "left")

def caesar_bruteforce(text):
    return [(k, caesar(text, k, "left")) for k in range(26)]

def caesar_frequency_guess(text):
    letters = clean(text)
    if not letters:
        return 0
    most_common = max(ALPHA, key=letters.count)
    return (ALPHA.index(most_common) - ALPHA.index("E")) % 26
```

Chiffrement de Hill

La clé est une matrice carrée modulo 26. Chaque bloc de n lettres est multiplié par la matrice. Pour déchiffrer on utilise l'inverse de la matrice, qui n'existe que si le déterminant est premier avec 26. L'attaque à clair connu retrouve la clé par K = C x P^-1.

```
def det(m):
    if len(m) == 1:
        return m[0][0]
    total = 0
    for j in range(len(m)):
        minor = [row[:j] + row[j + 1:] for row in m[1:]]
        total += (-1) ** j * m[0][j] * det(minor)
    return total

def mat_inverse_mod(m):
```

```
    n = len(m)
    d = det(m) % 26
    if gcd(d, 26) != 1:
        raise ValueError("la matrice clé n'est pas inversible modulo 26")
    d_inv = pow(d, -1, 26)
    if n == 1:
        return [[d_inv]]
    inverse = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j + 1:] for k, row in enumerate(m) if k != i]
            cofactor = (-1) ** (i + j) * det(minor)
            inverse[j][i] = cofactor * d_inv % 26
    return inverse

def check_hill_key(key):
    mat_inverse_mod(key)
    identity = all(key[i][j] % 26 == (1 if i == j else 0) for i in range(len(key)) for j in range(len(key)))
    if identity:
        raise ValueError("clé invalide : la matrice identité ne change pas le message")

def key_from_text(word):
    letters = clean(word)
    size = int(len(letters) ** 0.5)
    if size < 2 or size * size != len(letters):
        raise ValueError("la clé doit avoir 4 ou 9 lettres")
    numbers = [ALPHA.index(c) for c in letters]
    return [numbers[i * size:(i + 1) * size] for i in range(size)]

def hill_apply(text, key):
    n = len(key)
    numbers = [ALPHA.index(c) for c in text]
    result = ""
    for start in range(0, len(numbers), n):
        block = numbers[start:start + n]
        for row in key:
            result += ALPHA[sum(a * b for a, b in zip(row, block)) % 26]
    return result

def hill_encrypt(text, key):
    mat_inverse_mod(key)
    letters = clean(text)
    while len(letters) % len(key) != 0:
        letters += "X"
    return hill_apply(letters, key)

def hill_decrypt(text, key):
    inverse = mat_inverse_mod(key)
    letters = clean(text)
    if len(letters) % len(key) != 0:
        raise ValueError("la longueur du chiffré doit être un multiple de %d" % len(key))
    return hill_apply(letters, inverse)

def hill_strip_padding(text, n):
    removed = 0
    while text.endswith("X") and removed < n - 1:
        text = text[:-1]
        removed += 1
    return text

def block_matrix(text, start, n):
    numbers = [ALPHA.index(c) for c in text[start:start + n * n]]
    return [[numbers[i * n + j] for i in range(n)] for j in range(n)]

def mat_mul_mod(a, b):
    size = len(a)
    return [[sum(a[i][k] * b[k][j] for k in range(size)) % 26 for j in range(size)] for i in range(size)]

def hill_known_plaintext(plain, cipher, n=2):
    plain = clean(plain)
    cipher = clean(cipher)
    if len(plain) < n * n or len(cipher) < n * n:
        raise ValueError("il faut au moins n*n lettres")
    for start in range(0, len(plain) - n * n + 1, n):
```

```
        P = block_matrix(plain, start, n)
        C = block_matrix(cipher, start, n)
        try:
            return mat_mul_mod(C, mat_inverse_mod(P))
        except ValueError:
            continue
    raise ValueError("aucun bloc de clair inversible trouvé")
```

Chiffrement de Playfair

On construit un carré 5x5 à partir du mot clé (J = I). Le texte est coupé en paires (un X sépare les lettres doubles ou complète la fin). Même ligne : lettre de droite, même colonne : lettre du dessous, sinon on prend les coins du rectangle. Le déchiffrement fait l'inverse. digram_counts sert à l'attaque par fréquence des paires.

```
def check_playfair_key(key):
    if len(set(clean(key))) < 4:
        raise ValueError("clé invalide : il faut au moins 4 lettres différentes")

def playfair_square(key):
    letters = []
    for ch in clean(key).replace("J", "I") + ALPHA.replace("J", ""):
        if ch not in letters:
            letters.append(ch)
    return letters

def playfair_pairs(text):
    letters = clean(text).replace("J", "I")
    pairs = []
    i = 0
    while i < len(letters):
        first = letters[i]
        second = letters[i + 1] if i + 1 < len(letters) else "X"
        if first == second:
            second = "X"
            i += 1
        else:
            i += 2
        pairs.append((first, second))
    return pairs

def playfair_swap(a, b, square, step):
    row_a, col_a = divmod(square.index(a), 5)
    row_b, col_b = divmod(square.index(b), 5)
    if row_a == row_b:
        col_a = (col_a + step) % 5
        col_b = (col_b + step) % 5
    elif col_a == col_b:
        row_a = (row_a + step) % 5
        row_b = (row_b + step) % 5
    else:
        col_a, col_b = col_b, col_a
    return square[row_a * 5 + col_a] + square[row_b * 5 + col_b]

def playfair_encrypt(text, key):
    square = playfair_square(key)
    return "".join(playfair_swap(a, b, square, 1) for a, b in playfair_pairs(text))

def playfair_decrypt(text, key):
    square = playfair_square(key)
    letters = clean(text).replace("J", "I")
    if len(letters) % 2 != 0:
        raise ValueError("la longueur du chiffré doit être paire")
    return "".join(playfair_swap(letters[i], letters[i + 1], square, -1) for i in range(0, len(letters), 2))

def playfair_strip_padding(text):
    result = ""
    for i, ch in enumerate(text):
        between_equal = 0 < i < len(text) - 1 and text[i - 1] == text[i + 1]
        if ch == "X" and between_equal:
            continue
        result += ch
```

```
    if result.endswith("X"):
        result = result[:-1]
    return result

def digram_counts(text):
    letters = clean(text)
    counts = {}
    for i in range(0, len(letters) - 1, 2):
        pair = letters[i:i + 2]
        counts[pair] = counts.get(pair, 0) + 1
    return sorted(counts.items(), key=lambda item: -item[1])
```